

Sincronización OT ↔ Calendario — política

SF-659 · jornada VSC 2026-05-08 + reunión 2026-05-11

1. Fuente de verdad

La OT es la fuente de verdad. Todas las vistas de calendario (Weekly Schedule, Cronológico, Gantt View, Horarios, Technicians) leen de `managed_work_orders` y derivan su propio renderizado.

Cuando un usuario muta desde una vista de calendario (drag drop, click reschedule, edit fecha), el cambio se persiste vía endpoint dedicado (`/managed-work-orders/{id}` PUT o `/schedule` POST) y todas las demás vistas re-renderizan al recibir el broadcast WS o en su próximo poll.

2. Matriz de campos sincronizados

Campo OT	Calendar use	Mutable desde
<code>`planned_start`</code>	posición horizontal/horaria	Planning modal · Drag drop calendario · Auto-Schedule
<code>`planned_end`</code>	duración visual (derivado o explícito)	Planning modal · Drop con <code>OT.estimated_hours</code> · Manual edit
<code>`estimated_hours`</code>	duración intrínseca	Planning modal · derivado de ops
<code>`assigned_workers`</code>	tarjeta en fila técnico	Drag drop a fila técnico · Smart Assignment · OT modal
<code>`shift`</code> (DAY/NIGHT)	bucket día/noche	Implícito por <code>planned_start</code> hora · Drag drop a slot shift
<code>`status`</code> (PROGRAMADO/EN_EJECUCION/CERRADO)	color/visibilidad	Botones acción OT · Auto-transition
<code>`priority_code`</code>	badge color	OT modal (manual, SF-681)
<code>`wo_type`</code>	badge tipo	Sólo lectura (set al crear)
<code>`equipment_tag`</code>	tooltip / tarjeta	OT modal
<code>`planning_group`</code> / <code>`work_center`</code>	filtros calendario	OT modal

3. Reglas de conflictos

3.1 Concurrent edits (dos usuarios)

- **Optimistic concurrency** vía `version` column en `managed_work_orders` (SF-562).
- Cliente envía If-Match: `<version>` en PUT. Server compara; si difiere → 409 Conflict.
- Frontend muestra toast "Otro usuario modificó esta OT — recarga y reintenta".
- Excepción: `support_equipment` y `materials.reservation_code` bypass optimistic lock (metadata accesoria, no crítica — Jorge 2026-04-28).

3.2 Cambio de estado durante edición

- Si la OT pasa de PROGRAMADO a EN_EJECUCION mientras un usuario está editando fechas → el PUT falla con 409 + mensaje "OT ya está en ejecución, no se puede reschedule sin pasar por reschedule formal".
- Cambios de estado intermedios siempre van por endpoints dedicados (`/schedule` , `/start` , `/complete` , `/close` , `/reschedule`) que validan transiciones permitidas en `_STATUS_ORDER` (`managed_wo_service.py:857`).

3.3 OT sin fechas

- Si `planned_start` es NULL y la OT está en LIBERADO/PLANIFICADO → aparece en panel "OTs a Programar" del Scheduling.
- Drop al calendario asigna `planned_start = slot.timestamp` , `planned_end = planned_start + estimated_hours` .
- Si la OT está PROGRAMADO sin `planned_start` → es OT huérfana (SF-657, endpoint `/managed-work-orders/orphans`).

3.4 Recursos sobrecargados

- Antes de drop al calendario: frontend valida `techHours[worker_id] + wo.estimated_hours/n_workers ≤ HOURS_PER_WEEK` .
- Si excede: toast warning + bloqueo de drop (B3-7 Jorge 2026-04-27 — bloqueo duro en Reservar Semana).
- Si el conflict aparece tras un cambio de turno o ausencia → endpoint `/orphans` lo detecta y banner avisa.

3.5 Reschedule

- Endpoint `/reschedule` requiere `reason` no-vacío (SF-578).
- Audit log registra reschedule con razón + autor.
- Reschedule no incrementa `version` para evitar lock vs edits paralelos.

4. Edge cases tests pendientes (a automatizar)

- [] OT sin `planned_start` aparece en panel "A programar" y no en grid
- [] OT en PROGRAMADO sin `assigned_workers` aparece como orphan
- [] Drag drop fast (dos clicks <500ms) no duplica la asignación
- [] Reschedule a fecha pasada se permite pero genera audit + warning
- [] 409 conflict con If-Match desfasado retorna mensaje claro al usuario
- [] Auto-Schedule respeta calendario individual de técnico (`shift_pattern`)
- [] OT con operaciones multi-disciplina asigna techs por skill mix

5. Vistas calendario — desfase conocido

Vista	Refresca por WS	Refresca por poll	Refresca por button
-------	-----------------	-------------------	---------------------

Weekly Schedule (Technicians/Recursos/Work Orders/Horarios)	Sí (`wo_*`, `wr_*`)	No	Yes (Auto-Schedule / Auto-Level / Clear)
Cronológico	Sí	No	No
Gantt View	Sí	No	No
Mass Change	Sí	No	Yes
HH Balance	Derivado de Weekly	—	—

Issue conocido (Tanda 0 backlog): si el usuario está en `/work-management?tab=planning` y crea un WR desde `/failure-capture standalone`, el WS broadcast llega pero la lista no refetch hasta hard reload. BUG-03 fix mitigated con `wr : created event dispatch dual (immediate + 1.5s — commit 4c4b041)`.

6. Próximos pasos

- [] Tests Pytest sobre `_STATUS_ORDER` (no reversiones excepto vía endpoint dedicado)
- [] Tests Playwright sobre drag-drop calendario → fetch nueva OT en frontend
- [] Endpoint `/managed-work-orders/sync-check?plant_id=...` que retorna inconsistencias detectadas (OTs en calendario sin entrada en BD, etc.)
- [] Telemetría: contar 409 Conflict / día por `plant_id` para detectar contention real